

Technische Universität Berlin

Agententechnologien in betrieblichen Anwendungen und der Telekommunikation



Bachelorarbeit

**Monte Carlo Tree Search Hybride
in Spielen mit unvollständiger Information**

Leo Pirker

Matrikelnummer: 456758

21.02.2026

Prüfer

Prof. Dr. Albayrak

Prof. Dr. Tai

Betreuer

Stefan Fricke

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen die den benutzten Quellen und Hilfsmitteln wörtlich oder inhaltlich entnommen sind, habe ich als solche kenntlich gemacht. Sofern KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die Art der Nutzung (z.B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst. IB / Stand: April 2025 Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 15. Februar 2023. *Amtsblatt* habe ich zur Kenntnis genommen. Ich erkläre weiterhin, dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Angaben zur Nutzung von KI-Tools:

Produktname, Hersteller	Version	Nutzung
Gemini, Google	Gemini 1.5 Flash	Sprachliche Überprüfung und Korrektur von Rechtschreibung, Grammatik sowie Unterstützung bei der Formatierung von LaTeX-Elementen.
DeepL, DeepL SE	DeepL (Web-Version)	Übersetzung der Zusammenfassung und eines Zitats

Berlin, 21.02.2026



Leo Pirker

Abstract

This paper investigated the extent to which Monte Carlo tree search hybrids affect games with partial information. Hybrids intervene in the MCTS search by increasing the quality or quantity of rollouts. In this paper, hybrids were analysed in games with partial information. The Partially Observable Monte Carlo Planning (POMCP) algorithm, which specialises in partial information, was combined with hybrids. Chess variants with varying complexities were considered, including Dark Pawn Chess, a game played on a 5×5 board with only pawns, Dark Los Alamos Chess, played on a 6×6 board without bishops, and Dark Chess on an 8×8 board. In all the games tested, hybrids were found with a significant increase in the win rate compared to the baseline. The evaluation termination hybrid achieved win rates of 61.6% in Dark Pawn Chess and Dark Los Alamos Chess against the baseline. This hybrid aborts rollouts early when a threshold is exceeded and evaluates the current game situation as won/lost. In Dark Chess, the promising abort hybrid proved most successful with a 74% win rate, which aborts after a fixed rollout depth and returns the result of a minimax search.¹

¹Die englischsprachige Zusammenfassung wurde unter Verwendung des KI-Tools DeepL erstellt und sprachlich angepasst.

Zusammenfassung

In dieser Arbeit wurde untersucht, inwiefern Monte-Carlo-Tree-Search-Hybride sich in Spielen mit partiellen Informationen auswirken. Hybride greifen dabei in die MCTS-Suche ein, indem sie die Qualität oder die Quantität der Rollouts steigern. Dabei wurden unter anderem Hybride untersucht, die auf Bewertungsfunktionen zurückgreifen. In dieser Arbeit wurden diese Hybride in Spielen mit partiellen Informationen analysiert. Dafür wurde der auf partielle Sicht spezialisierte Algorithmus Partially Observable Monte Carlo Planning (POMCP) mit den Hybriden kombiniert. Dabei sind Schachvarianten mit unterschiedlichen Komplexitäten betrachtet worden, Dark Pawn Chess ein Spiel auf einem 5×5 Spielfeld nur mit Bauern, Dark Los Alamos Chess auf einem 6×6 Spielfeld ohne Läufer und Dark Chess auf einem 8×8 Spielfeld.

In allen getesteten Spielen konnten Hybride gefunden werden mit einer signifikanten Steigerung der Siegesrate gegenüber der Baseline. Der Bewertungabbruch-Hybrid erzielte in Dark Pawn Chess und Dark Los Alamos Chess Siegesraten von 61,6%. Dieser bricht Rollouts bei dem Überschreiten eines Schwellenwertes frühzeitig ab und wertet die aktuelle Spielsituation als Gewonnen/Verloren. In Dark Chess zeigte sich der Vielversprechende Abbrüche Hybrid mit 74% Siegesrate am erfolgreichsten, welcher nach einer festen Rollout-Tiefe abbricht und das Ergebnis einer Minimax-Suche zurückliefert.

Inhaltsverzeichnis

Abbildungsverzeichnis	vii
Tabellenverzeichnis	viii
1 Einleitung	1
1.1 Motivation	1
1.2 Zielsetzung und Umfang der Arbeit	1
1.3 Gliederung der Arbeit	1
2 Spiele	3
2.1 Dark Pawn Chess	4
2.2 Dark Los Alamos Chess	5
2.3 Dark Chess	6
3 Monte-Carlo Tree Search mit unvollständigen Informationen	7
3.1 Monte-Carlo Tree Search	7
3.1.1 Upper Confidence Bounds applied to Trees	8
3.2 Partial Observable Monte Carlo Planning	8
3.2.1 Partikelfilter	8
3.2.2 Ablauf	9
3.3 MCTS-Hybride	10
3.3.1 Zugwahl-Hybride	10
3.3.2 Abbruchs-Hybride	11
3.4 Zusammenfassung	12
4 Implementierung	13
4.1 Projekt-Modellierung	13
4.1.1 Schnittstelle ChessVariation	13
4.1.2 Spielserver	14
4.1.3 KIs	14
4.2 Bitboards	14
4.3 Statische Speicherallokation	16
4.4 Partikelfilter	16
4.5 Bewertungsfunktionen	17
4.5.1 Dark Pawn Chess	17
4.5.2 Dark Los Alamos Chess und Dark Chess	17
4.6 Gegnermodellierung	18

5	Experimente	20
5.1	Dark Pawn Chess	20
5.2	Dark Los Alamos Chess	23
5.3	Dark Chess	26
5.4	Analyse der Hybride in den verschiedenen Spielen	29
6	Fazit	31
6.1	Zusammenfassung	31
6.2	Probleme	31
6.3	Ausblick	32
	Literaturverzeichnis	33

Abbildungsverzeichnis

2.1	Die untersuchten Schachspiele in ihren Startaufstellungen. Von links nach rechts: Dark Pawn Game, Dark Los Alamos Chess, Dark Chess. Quelle: Eigene Darstellung	4
2.2	Die Sicht der weißen Spielerin auf Dark Pawn Game. Felder außerhalb ihrer Wahrnehmung sind mit einem blauen Kreuz markiert. Dabei fällt auf, dass gerade im Mittelspiel (rechte Abbildung) nur wenige Felder außerhalb der Sicht sind. Quelle: Eigene Darstellung	5
2.3	Die Sicht der Schwarzen Spielerin auf ein Beispielspielzustand. Quelle: Eigene Darstellung	5
4.1	UML-Klassendiagramm der Implementierung. Quelle: Eigene Darstellung .	13
4.2	Bitboard-Repräsentation eines Schachbretts. Quelle: In Anlehnung an [Ras04]	15
4.3	Mögliche Züge für den Springer auf B1 blau markiert, der Springer kann nicht auf D2 ziehen da das Feld durch eine eigene Figur blockiert wird. Quelle: Eigene Darstellung	15
4.4	Weiß am Zug und untersucht die Aktion C3-B5. Quelle: Eigene Darstellung	18
5.1	Siegesraten des Schlagpräferenz-Hybrid (a), des K-Beste-Hybrid (b), Greedy-Hybrid (c) und Korrektur-Hybrid (d), unter Variation ihrer Parameter. Quelle: Eigene Darstellung	21
5.2	Siegesraten des Vielversprechende-Rollouts (a), des Vielversprechende-Abbrüche-Hybrid (b), Frühzeitige-Abbrüche-Hybrid (c) und Bewertungs-Abbruch-Hybrid (d), unter Variation ihrer Parameter. Quelle: Eigene Darstellung .	22
5.3	Siegesraten des Vielversprechende-Rollouts (a), des Vielversprechende-Abbrüche-Hybrid (b), Frühzeitige-Abbrüche-Hybrid (c) und Bewertungs-Abbruch-Hybrid (d), unter Variation ihrer Parameter. Quelle: Eigene Darstellung .	24
5.4	Siegesraten des Vielversprechende-Rollouts (a), des Vielversprechende-Abbrüche-Hybrid (b), Frühzeitige-Abbrüche-Hybrid (c) und Bewertungs-Abbruch-Hybrid (d), unter Variation ihrer Parameter. Quelle: Eigene Darstellung .	25
5.5	Siegesraten des Vielversprechende-Rollouts (a), des Vielversprechende-Abbrüche-Hybrid (b), Frühzeitige-Abbrüche-Hybrid (c) und Bewertungs-Abbruch-Hybrid (d), unter Variation ihrer Parameter. Quelle: Eigene Darstellung .	27
5.6	Siegesraten des Vielversprechende-Rollouts (a), des Vielversprechende-Abbrüche-Hybrid (b), Frühzeitige-Abbrüche-Hybrid (c) und Bewertungs-Abbruch-Hybrid (d), unter Variation ihrer Parameter. Quelle: Eigene Darstellung .	28

Tabellenverzeichnis

3.1	Die Hybride im Überblick	12
5.1	Die Siegesraten der optimierten Hybride in den drei getesteten Spielen . .	29
5.2	Der Median der Rollouts/s der optimierten Hybride in den drei getesteten Spielen	30

1 Einleitung

1.1 Motivation

Schachvarianten mit unvollständigen Informationen stellen eine besondere Herausforderung in der Spieltheorie dar. Im Gegensatz zum klassischen Schach ist hier nicht das gesamte Spielfeld einsehbar, die Positionen der gegnerischen Figuren sind teilweise unbekannt. Ziel dieser Arbeit ist es, eine Reihe von Algorithmen, die bereits erfolgreich in anderen Spielen eingesetzt worden sind, in Schachvarianten verschiedener Komplexitäten zu implementieren und zu vergleichen.

Die untersuchten Schachvarianten sind (in aufsteigender Komplexität): Dark Pawn Game (5×5), Dark Los Alamos Chess (6×6) und das klassische Dark Chess (8×8).

1.2 Zielsetzung und Umfang der Arbeit

Als Grundlage aller untersuchten KIs dient der Partially Observable Monte Carlo Planning-Algorithmus (POMCP). Eine Schwäche von POMCP ist, dass bei der Bewertung eines Spielzustandes ab einem gewissen Punkt in sogenannten Rollouts beide Seiten zufällige Züge ausführen, bis eine Seite gewonnen hat. Das große Problem daran ist, dass diese Rollouts bei komplexen Spielen sehr lang sein können und das Ergebnis unter Umständen nur schwach mit dem untersuchten Zustand korreliert ist: Selbst wenn der ursprüngliche Zustand beispielsweise sehr vorteilhaft für Weiß war, ist es durch die zufällige Zugwahl nicht unwahrscheinlich, dass im Rollout Schwarz gewinnt. Zusätzlich zu ihrer geringen Aussagekraft ist ein weiteres Problem der Rollouts ihr hoher Rechenaufwand.

Das primäre Ziel dieses Projektes ist die Implementierung und Evaluierung eines MCTS-basierten Agenten für Spiele mit partieller Beobachtbarkeit. Als Grundlage dient der *Partially Observable Monte Carlo Planning*-Algorithmus (POMCP). Dieser soll im Rahmen der Arbeit zu verschiedenen Hybriden erweitert werden, um spezifische Schwächen klassischer Rollouts zu adressieren.

1.3 Gliederung der Arbeit

In Kapitel 2 werden die untersuchten Spiele vorgestellt. Dabei wird auf die partielle Sicht eingegangen, die Spielregeln vorgestellt und die Komplexität der Spiele diskutiert. Im Anschluss wird in Kapitel 3 ein Überblick über die genutzten Algorithmen verschafft. Dabei wird die Monte-Carlo-Tree-Search eingeführt und erläutert, inwiefern „Partially Observable Monte Carlo Planning“ in Umgebungen mit partieller Sicht angewendet wird.

Außerdem werden die Funktionsweise der genutzten Hybride vorgestellt. In Kapitel 4 wird auf die Implementierung eingegangen. Im Anschluss stellt Kapitel 5 die durchgeführten Experimente vor und wertet diese aus. Als Abschluss wird dann in Kapitel 6 die Arbeit zusammengefasst und geht auf die Probleme ein, welche während der Arbeit auftraten und bietet einen Ausblick auf zukünftige Projekte.

2 Spiele

Untersucht werden hybride MCTS-Algorithmen in drei Schachversionen (siehe Abb. 2.1) mit unvollständigen Informationen. In dieser Arbeit wird nicht näher auf die Zugmuster der Figuren eingegangen. Diese finden Sie in den Schachregeln [Koh26].

Die untersuchten Spiele sind von unterschiedlicher Komplexität. Die Komplexität in Spielen mit perfekten Informationen wird bestimmt durch den durchschnittlichen Verzweigungsgrad (durchschnittliche Anzahl an verfügbaren Zügen pro Zustand) und die durchschnittliche Anzahl an Zügen [A⁺94].

Die optimale Strategie bei Spielen mit perfekter Information ist so definiert, dass „einseitiges Abweichen“ von dieser Strategie dem Gegner keinen Vorteil verschaffen darf (sogenanntes Nash-Gleichgewicht). Das entspricht insofern unserer Intuition, da wir unseren Zug so wählen müssen, dass wir den Gegner nicht in eine Lage bringen, in der er einen für ihn vorteilhaften Zug auswählen kann. Bei Spielen mit vollständiger Information ist die Bewertung jedes Spielzustandes eindeutig, eine Schachstellung ist also, wenn mit optimaler Strategie weitergespielt wird, aus der Perspektive einer bestimmten Partei entweder Sieg, Niederlage oder Unentschieden.

Dies ist in Spielen mit unvollständigen Informationen anders. Allis untersuchte die Komplexität von Spielen und den Unterschied zwischen Spielen mit vollständigen Informationen und unvollständigen: *„Das optimale Spiel in einem Spiel mit perfekter Information besteht immer aus einer reinen Strategie, während in Spielen mit unvollständiger Information das optimale Spiel eine gemischte Strategie erfordern kann. In einer reinen Strategie kann für jeden Spielzustand ein einziger Zug bestimmt werden, der zum spieltheoretischen Wert der Position führt. Bei einer gemischten Strategie erfordert das optimale Spiel, dass eine Spielerin einen Zug i mit der Wahrscheinlichkeit p_i spielt, wobei mindestens zwei solcher p_i ungleich Null sind.“*¹ [A⁺94]

Die gemischte Strategie erfordert also, dass wir mit dem bestehenden Wissen, welches wir über die Stellung haben, zufällig zwischen mehreren Zügen wählen, um für unseren Gegner unberechenbar zu bleiben. Entsprechend müssen auch wir eine Strategie wählen, welche möglichst robust auf alle möglichen gegnerischen Züge antwortet.

Ein offensichtliches Beispiel für die Notwendigkeit einer gemischten Strategie ist die Eröffnung: Die weiße Spielerin hat perfekte Informationen über das Brett, trotzdem ist es unvorteilhaft, immer die gleiche Eröffnung zu spielen, selbst wenn diese bei einem Spiel mit vollständigen Informationen optimal wäre, um der schwarzen Spielerin keine Infor-

¹Die deutsche Übersetzung wurde unter Verwendung des KI-Tools DeepL erstellt und sprachlich angepasst.

mationen zu geben.

Dadurch gibt es keine eindeutige Bewertung mehr für eine Spielsituation, sondern nur eine statistische Wahrscheinlichkeit für den Ausgang.

Von praktischem Nutzen ist für uns außerdem: Um die optimale gemischte Strategie zu finden, darf unser statistischer Algorithmus bei hoher Laufzeit nicht immer zum gleichen Ergebnis konvergieren.

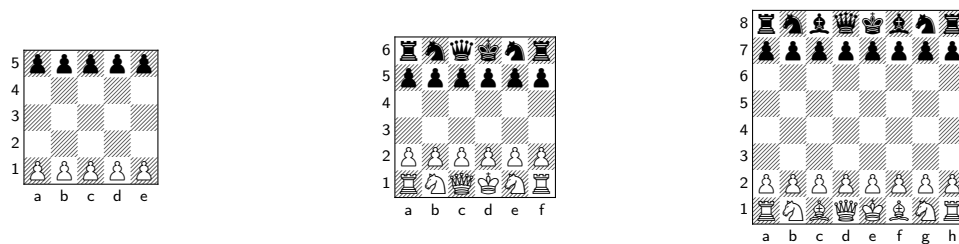


Abbildung 2.1: Die untersuchten Schachspiele in ihren Startaufstellungen. Von links nach rechts: Dark Pawn Game, Dark Los Alamos Chess, Dark Chess. Quelle: Eigene Darstellung

2.1 Dark Pawn Chess

Dark Pawn Chess ist ein Spiel auf einem 5×5 großen Schachbrett. Bei Spielbeginn stehen sich auf den Grundlinien die Bauern beider Spielerinnen gegenüber. Weiß beginnt das Spiel, die Spielerin, die am Zug ist, muss einen ihrer Bauern nach vorne ziehen. Die Sicht einer Spielerin ist begrenzt auf die eigenen Bauern und die Felder, auf denen diese ihren Zug beenden können. D. h., Bauern nehmen immer das Feld vor sich sowie links diagonal und rechts diagonal vor sich wahr. Falls diagonal vor dem Bauern eine gegnerische Figur steht, kann diese Figur von der Spielerin geschlagen werden. Die Spielerin, die einen Bauern auf die Grundlinie der Gegnerin zieht, gewinnt das Spiel.

Sollte eine Spielerin, die am Zug ist, keine möglichen Züge haben, so endet das Spiel. Es gewinnt dann die Spielerin mit den meisten Bauern auf dem Spielbrett. Bei gleicher Anzahl an Bauern endet das Spiel im Remis. Dark Pawn Chess ist ein Spiel mit einem geringen Zustandsraum und einem geringen Verzweigungsgrad. Dadurch, dass mit jedem Zug Bauern in Richtung der gegnerischen Grundlinie gezogen werden müssen, ist auch die Spieldauer kurz. Und da das Spielfeld so klein ist, sieht eine Spielerin oft große Teile desselben (siehe Abb. 2.2), die Anzahl der aus ihrer Perspektive möglichen Spielzustände, die sie berücksichtigen muss, ist also relativ klein.



Abbildung 2.2: Die Sicht der weißen Spielerin auf Dark Pawn Game. Felder außerhalb ihrer Wahrnehmung sind mit einem blauen Kreuz markiert. Dabei fällt auf, dass gerade im Mittelspiel (rechte Abbildung) nur wenige Felder außerhalb der Sicht sind. Quelle: Eigene Darstellung

2.2 Dark Los Alamos Chess

Dark Los Alamos Chess ist eine Schachversion auf einem 6×6 Spielfeld (siehe Abb. 2.3). Dabei wird ohne Läufer gespielt. Auch hier ist die Sicht begrenzt auf die Figuren der Spielerin und die Felder, auf die diese ziehen können. Es gewinnt die Spielerin, die den gegnerischen König schlägt. Los Alamos Chess wird ohne Doppelzug von unbewegten Bauern gespielt, damit entfällt auch das damit verbundene en passant. Auch wird ohne Rochade gespielt. Wenn ein Bauer die gegnerische Grundlinie erreicht, wird dieser in eine Dame umgewandelt.

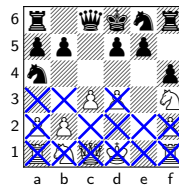


Abbildung 2.3: Die Sicht der schwarzen Spielerin auf einen Beispielspielzustand. Quelle: Eigene Darstellung

Falls ein Spiel nach über 200 Zügen noch nicht beendet ist, wird dieses abgebrochen und als Unentschieden gewertet. Dark Los Alamos Chess besitzt einen kleinen Zustandsraum, aber einen Verzweigungsgrad ähnlich wie bei Schach. Da nur die zwei mittleren Reihen anfangs unbesetzt sind, hat Weiß einen großen Startvorteil. Denn Schwarz sieht nach dem ersten Zug von Weiß deren Bauern im Zentrum nicht und kann nur unter Gefahr im ersten Zug die Springer ins Spiel bringen. Durch das kleine Spielfeld und die vielen Figuren stehen den Spielerinnen viele Informationen zur Verfügung. Die Anzahl der möglichen Spielsituationen ist im Start- und Mittelspiel klein.

2.3 Dark Chess

Für Dark Chess gelten dieselben Regeln wie für Dark Los Alamos Chess, mit dem Unterschied, dass auf einem 8×8 Spielfeld gespielt wird, und mit Läufern. Außerdem können die Bauern von ihrer Startposition aus zwei Felder nach vorne ziehen. Dark Chess besitzt einen großen Zustandsraum und einen hohen Verzweigungsgrad. Durch die vier unbesetzten Reihen in der Mitte des Spielfeldes wird auch die Informationsbeschaffung schwierig. Dadurch gibt es eine große Menge an möglichen Spielzuständen.

3 Monte-Carlo Tree Search mit unvollständigen Informationen

Die untersuchten Spiele starten in einem festen Startzustand, wobei Weiß beginnt. Es wird so lange eine mögliche Aktion ausgewählt, bis ein Endzustand erreicht wird. Dabei ist nach jedem Zug die andere Spielerin an der Reihe. Ein Endzustand ist erreicht, wenn entweder keine Aktion mehr zur Verfügung steht oder das Spielende erreicht ist. Ein Spiel kann entweder von einer der Parteien gewonnen werden oder als Unentschieden (Remis) enden. Algorithmen wie Minimax und Monte Carlo Tree Search werden eingesetzt, um vielversprechende Aktionen zu finden. Dabei simulieren die Algorithmen Spielverläufe, bis ein Endzustand erreicht wird. Alternativ kann eine Simulation abgebrochen werden, wenn sie zu lang wird. Der erreichte Zustand wird dann durch eine Bewertungsfunktion bewertet.

Diese Bewertung beeinflusst, wie attraktiv wir die Aktionen einschätzen, die uns zu diesem Zustand gebracht haben.

Am Ende der Programmausführung wird die vielversprechendste Aktion ausgewählt. Dies ist die Aktion, die in den bestbewertete Nachfolgezustand übergeht.

Verfügt man bei einem bestimmten Spiel über Domänenwissen, kann man dieses mithilfe einer **Bewertungsfunktion** nutzen. Diese bewerten i. d. R. die Spielsituation nicht exakt (das würde bedeuten, dass sie erkennt, ob das Spiel gewonnen oder verloren ist), sondern geben nur eine Näherung in Form einer Heuristik. Bewertungsfunktionen bei Schach berücksichtigen normalerweise: Material (gewichtet nach Figurentypen), Bauernstruktur, Mobilität der Figuren und Zentrumskontrolle [Ras04]. Die in diesem Projekt genutzte Bewertungsfunktion wird in Kapitel 4 besprochen.

3.1 Monte-Carlo Tree Search

Coulom stellte die Monte-Carlo Tree Search (MCTS) 2006 vor [Cou06], ein nichtdeterministisches Suchverfahren, um die beste Aktion in einer vollständig sichtbaren Umgebung zu finden. Er implementierte den Algorithmus als KI für das chinesische Brettspiel Go, welches auf dem 9×9 -Feld ein Turnier zwischen KIs gewinnen konnte. MCTS wird in vier Phasen unterteilt, diese werden bis zum Erreichen eines Abbruchkriteriums (z. B. Erreichen von n -Rollouts oder abgelaufene Zeit) wiederholt. Jeder Knoten des Suchbaums speichert die Anzahl der Besuche und wie oft diese Spiele in einem Sieg enden.

Selection: Startet von der Wurzel des Baumes (entspricht der betrachteten Spielsituation) und wählt Aktionen, bis ein Blattknoten erreicht ist. Wir nutzen Upper Confidence Bounds (UCT), um eine vielversprechende Aktion zu wählen. Bei Zügen des Gegners wird diese Formel vergleichbar zu Minimax angepasst. Das bedeutet, dass in diesen Knoten wahrscheinlich Züge ausgewählt werden, die eine hohe Siegesrate für den Gegner haben.

Expansion: Beim Erreichen eines Blattknotens wird zufällig eine hier verfügbare Aktion ausgeführt. Der Zustand, der unter Ausführung der gewählten Aktion erreicht wird, wird dem Baum hinzugefügt.

Rollout: Wählt so lange zufällige Aktionen aus, bis ein Endzustand erreicht wurde.

Backpropagation: Gibt den Spielausgang zurück und aktualisiert die durchlaufenen Knoten des Baumes. Dabei wird an den Knoten die Anzahl an Besuchen inkrementiert, und im Fall eines Sieges wird auch die Anzahl der Siege inkrementiert. Nach Abschluss von MCTS gibt der Algorithmus die Aktion mit der höchsten Siegesrate zurück.

3.1.1 Upper Confidence Bounds applied to Trees

Upper Confidence Bounds applied to Trees (UCT) trifft eine Abwägung zwischen Erkundung (Exploration) und Ausbeutung (Exploitation). Dabei sind w_i die Siege des Knotens, n_i die Besuche des Knotens, N_i die Besuche des Elternknotens und c ist eine Konstante. Bei einem Knoten, der bisher eine gute Siegesrate hatte, ist es wahrscheinlicher, dass er auch tatsächlich ein guter Zug ist, deshalb suchen wir hier verstärkt. Das entspricht dem Denkmuster einer Schachspielerin, länger über vielversprechende Züge nachzudenken.

$$UCT = \frac{w_i}{n_i} + c \sqrt{\frac{\log N_i}{n_i}} \quad (3.1)$$

Der rechte Teil der Formel ist für die breitere Erkundung des Suchbaumes zuständig. Er ist umso größer, je seltener der Knoten im Verhältnis zu den Besuchen des Elternknotens besucht wurde. Die Konstante c muss so gewählt werden, dass eine gute Abwägung zwischen Ausbeutung und Erkundung stattfindet. Dies ist aber domänenspezifisch.

3.2 Partial Observable Monte Carlo Planning

Partially Observable Monte Carlo Planning ist eine Weiterentwicklung von MCTS für Anwendungen mit partiellen Informationen, vorgestellt von Silver und Veness [SV10]. Dabei ergänzen die beiden MCTS um Beobachtungen und einen Partikelfilter.

3.2.1 Partikelfilter

Da wir uns in Spielen mit partiellen Informationen befinden, wissen wir nicht, in welchem Spielzustand wir uns befinden. Denn gegnerische Figuren befinden sich eventuell nicht

in unserer Wahrnehmung. Eine Möglichkeit, dieses Problem zu adressieren, wäre es, alle möglichen Spielzustände zu speichern, die mit unserer Sicht auf das Spielfeld und dem bisherigen Spielverlauf kompatibel sind. Dies ist jedoch oft nicht praktikabel, weil die Anzahl möglicher Spielzustände besonders bei großen Spielbrettern exponentiell anwächst. Der Partikelfilter enthält eine vergleichsweise geringe Menge an möglichen Spielzuständen, wir müssen allerdings darauf achten, dass er so viele enthält, dass diese repräsentativ sind. In der Selection- und Expansionsphase speichern wir den Spielzustand zugeordnet an den untersuchte Knoten für den Fall, dass sich das tatsächliche Spiel in diese Richtung des Suchbaumes entwickelt, dann bilden diese Zustände den Partikelfilter des Knotens.

Bezüglich der Größe des Partikelfilters kann es sowohl passieren, dass er zu klein wird und die tatsächliche Vielfalt der möglichen Spielzustände nicht abbildet, als auch, dass er zu groß und verwaltungsaufwendig wird, wenn zu viele Zustände generiert werden. Ersteres tritt ein, wenn in einem POMCP-Durchlauf in der Wurzel nicht alle möglichen Aktionen durchprobiert werden. Silver und Veness sehen eine Funktion vor, die einem verwaisten Partikelfilter Spielzustände hinzufügt, indem bestehende Zustände des Partikelfilters genommen und durch Anwendung eines Rauschens ein neuer Zustand geschaffen wird, welcher wiederum dem Partikelfilter hinzugefügt wird.

3.2.2 Ablauf

POMCP wird gestartet indem die aktuellen Beobachtungen POMCP übergeben werden. Dies ist bei uns die Sicht der aktuellen Spielerin auf das Spielbrett. Der Partikelfilter wird so angepasst, dass sich nur noch mit den Beobachtungen konsistente Zustände in dem Partikelfilter befinden. Die Partikel im Partikelfilter des Wurzelzustand müssen überprüft werden, ob diese konsistent sind mit den Beobachtungen. Nur konsistente Beobachtungen bleiben im Partikelfilter. Danach zieht POMCP für jede Simulation ein zufälliges Partikel aus den Partikelfilter des Wurzelknotens. Und führt darauf die Phasen der MCTS aus. Die Knoten im Spielbaum von POMCP sind dabei mit $\langle h, a, o \rangle$ kodiert. Wobei h die Geschichte des Spiels ist, also alle bisherigen Aktionen und Beobachtungen, h ist also: $((a_0, o_0), \dots, (a_{t-1}, o_{t-1}))$, während a die aktuelle Aktion und o die aktuelle Beobachtung ist. Werden zwei verschiedene Partikel b_1 und b_2 aus dem Partikelfilter gezogen und auf beiden Aktion a_1 ausgeführt und anschließend Beobachtung o_1 gemacht, dann führen beide Partikel unter Aktion a_1 und Beobachtung o_1 in den gleichen Knoten $\langle h, a_1, o_1 \rangle$. Beim durchlaufen des Suchbaums wird der Partikelfilter des aktuellen Knoten erweitert um den aktuellen Spielzustand. Ist die Abbruchbedingung von POMCP erreicht wird die beste Aktion bestimmt und der Suchbaum beschnitten. Dabei wird der Knoten $\langle h, a, o \rangle$ zum neuen Wurzelknoten, bei gewählter Aktion a und neuer Beobachtung o . Behalten wird nur der Baum unterhalb von $\langle h, a, o \rangle$.

3.3 MCTS-Hybride

MCTS konnte erfolgreich zu sogenannten MCTS-Hybriden erweitert werden, sodass MCTS durch qualitativ hochwertigere oder kürzere Rollouts bessere Züge generiert. Dabei wird meist auf Bewertungsfunktionen und Minimax zurückgegriffen. Die Eigenschaft von MCTS, dass es ohne Domänenwissen funktioniert, wird aufgegeben, und eine gute Heuristik wird von zentraler Bedeutung.

Minimax ist ein Suchalgorithmus, der einen vollständigen Suchbaum bis zu einer bestimmten Tiefe generiert. Das bedeutet, dass von einem Startzustand aus alle verfügbaren Knoten generiert werden, auf den generierten Kindknoten wird der Algorithmus bis zum Erreichen der Suchtiefe rekursiv aufgerufen. Wird die maximale Suchtiefe erreicht, bewertet eine Bewertungsfunktion den Zustand. Endzustände werden mit 1 (Sieg Weiß), 0 (Remis) oder -1 (Sieg Schwarz) bewertet. Beim Zurückpropagieren wird jeweils die optimale Aktion gewählt: Die minimierende Spielerin wählt den kleinsten Wert, die maximierende Spielerin den größten Wert.

Wir teilen die innerhalb dieser Arbeit genutzten Hybride in Zugwahl- und Abbruchs-Hybride ein.

3.3.1 Zugwahl-Hybride

Zugwahl-Hybride kommen in der Zugauswahl während des MCTS-Rollouts zum Einsatz. Die Aktionen werden also nicht mehr per Zufall ausgewählt, sondern durch die Strategie des Hybriden. Die Berechnung der Strategie verlangsamt den Rollout, soll aber durch ihre Zugauswahl die Qualität des Rollouts verbessern.

Zugsortierung und k-beste Züge

Die möglichen Aktionen werden durch eine Heuristik bewertet. Danach wird die Zugauswahl durch eine Zugsortierung und ein Beschneiden auf die k-besten Züge begrenzt. Im Anschluss daran wird eine der verbleibenden Aktionen zufällig gleichverteilt ausgewählt[BW14]. Zugsortierung und k-beste Züge kürzen wir auch mit k-beste ab.

Schlagpräferenz

Mit einer Wahrscheinlichkeit α wird ein Schlagzug ausgewählt. Schlagzüge sind dabei diejenigen Züge, bei denen eine gegnerische Figur geschlagen wird. Wird kein Schlagzug ausgeführt, wird ein zufälliger Zug gewählt.

Korrektur

Korrektur nutzt eine Bewertungsfunktion, um den aktuellen Spielzustand sowie alle ihrer Folgezustände zu bewerten [WB09]. Wird ein Folgezustand schlechter bewertet als der aktuelle, wird dem Zustand ein konstanter Wert ϵ nahe null zugewiesen. Schlussendlich

wird dann ein Zug gewichtet, zufällig nach seiner Bewertung, ausgewählt und an MCTS zurückgegeben. Wird ein spielgewinnender Zug gefunden, wird dieser sofort zurückgegeben.

Greedy

Greedy wählt bevorzugt die Aktion, die laut Bewertungsfunktion in den bestbewerteten Folgezustand übergeht [WB09]. Um Determinismus zu vermeiden, besitzt der Greedy-Hybrid einen Parameter. Dieser Parameter gibt an, mit welcher Wahrscheinlichkeit der optimale Zug gewählt werden soll, ansonsten wird ein zufälliger Zug zurückgeliefert.

Vielversprechende Züge

Vielversprechende Züge ist ein Hybrid, der zwei Parameter besitzt. Die Idee ist, dass die Aussagekraft des Rollouts gesteigert wird, indem von Zeit zu Zeit eine Minimax-Suche gestartet wird [BW14]. Wird keine Minimax-Suche gestartet, wird ein zufälliger Zug gewählt. Die Wahrscheinlichkeit, dass für eine Zugwahl eine Minimax-Suche gestartet wird, und die entsprechende Suchtiefe sind die Parameter des Hybriden.

3.3.2 Abbruchs-Hybride

Abbruchs-Hybride brechen den MCTS-Rollout nach einer festgelegten Regel ab, mit dem Ziel, kürzere, aber dafür mehr Rollouts durchzuführen. Diese Hybride setzen eine Bewertungsfunktion ein, um den erreichten Spielzustand zu bewerten, und propagieren diesen durch den Spielbaum zurück.

Bewertungs-Abbruch

Für den Bewertungs-Abbruch-Hybrid wird ein Schwellwert festgelegt, ab dem ein Zustand des Spiels als gewonnen gilt [WB09]. Übersteigt die Bewertung des aktuellen Zustands diesen Wert, wird der Rollout abgebrochen. Dasselbe gilt auch für eine Niederlage. Die für sicher angenommene Sieg- oder Niederlage wird dann zurückpropagiert.

Frühzeitiger-Abbruch

Beim Erreichen einer festgelegten Suchtiefe d wird der MCTS-Rollout abgebrochen. Der erreichte Zustand wird bewertet, dieser Wert wird zurückgegeben [Lor15].

Vielversprechende Abbrüche

Bei festgelegter Tiefe wird der Rollout abgebrochen. Auf dem Endzustand wird eine Minimax-Suche gestartet, deren Ergebnis wird zurückpropagiert [BW14].

3.4 Zusammenfassung

MCTS approximiert die beste Aktion unter vollständigen Informationen. POMCP wendet MCTS in Umgebungen mit unvollständigen Informationen an. Dabei behält POMCP in einem Partikelfilter mögliche Zustände des aktuellen Knotens. Vor jeder MCTS-Simulation wird ein Spielzustand zufällig aus dem Partikelfilter gezogen. MCTS konnte schon mit anderen Algorithmen zu MCTS-Hybriden kombiniert werden, um höhere Siegesraten zu erzielen als reines MCTS. Innerhalb dieser Arbeit wird untersucht, ob Hybride unter Nutzung einer Bewertungsfunktion, von Minimax oder Parametern wie Schlagpräferenz bessere Resultate als POMCP erzielen können. Einen Überblick über die untersuchten Hybride bietet Tabelle 3.1.

Hybrid	Minimax	Bewertungsfunktion
Korrektur	-	ja
Greedy	-	ja
Bewertungs-Abbruch	-	ja
Frühzeitiger-Abbruch	-	ja
Vielversprechende-Züge	ja	ja
Vielversprechende-Abbrüche	ja	ja
Schlagpräferenz	-	-
Zugsortierung und k-beste	-	ja

Tabelle 3.1: Die Hybride im Überblick

4 Implementierung

Da die Geschwindigkeit des Programmes im Vordergrund steht, wurde dieses Projekt in der Programmiersprache Golang umgesetzt. Der Quellcode dieses Projektes befindet sich im GitLab der TU-Berlin: ¹.

4.1 Projekt-Modellierung

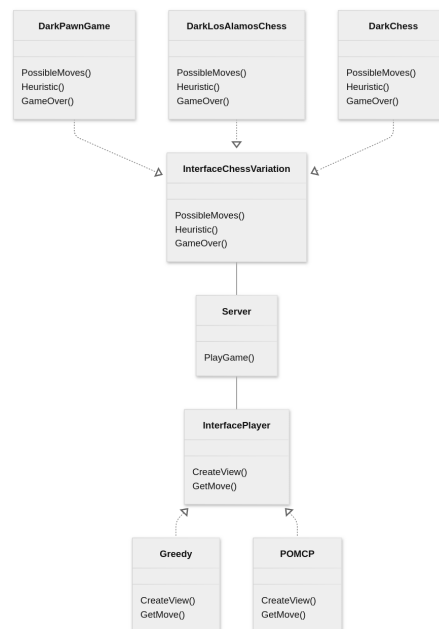


Abbildung 4.1: UML-Klassendiagramm der Implementierung. Quelle: Eigene Darstellung

Innerhalb des Projektes wurden mehrere verschiedene Spiele, KIs und Hybride umgesetzt. Um die Komponenten einfach tauschen zu können, wurden Schnittstellen eingesetzt. Damit wird auch die Interoperabilität der KIs mit den Spielen und den Hybriden sichergestellt.

4.1.1 Schnittstelle ChessVariation

Die Schachspiele implementieren die Schnittstelle `ChessVariation` und implementieren dadurch die folgenden Funktionen:

¹<https://git.tu-berlin.de/lep333/montecarlotheresearchhybride>

- `GameOver()`: Überprüft ob ein Spiel entschieden ist.
- `Heuristic()`: bewertet den aktuellen Spielzustand
- `PossibleMoves()`: gibt ein Array zurück, welches alle möglichen Züge im aktuellen Spielzustand enthält
- `ViewHash()` und `Hash()`: geben das aktuelle Spielfeld als **Zobrist Hash**[Zob90] zurück. Um Zobrist Hashes zu erstellen wird für jede Position, jeden Figurentyp und jede Spielerin eine Zufallszahl erstellt. Für jede auf dem Spielbrett befindliche Figur wird dann die zugeordnete Zufallszahl mit dem XOR Operator verknüpft. Bei 64-Bit Zahlen ist dann die Chance für Hash-Collisions minimal. `ViewHash()` benutzt dabei nur Felder zum Hashen die die Spielerin auch sehen kann
- `ExecuteMove(move)`: führt den übergebenen Zug durch
- `ReturnBoard()`: gibt eine Kopie des Spielbretts zurück

4.1.2 Spielserver

Der Server bekommt bei der Initialisierung Spiel und Spieler übergeben. Der Server ruft die Spielerinnen in Reihenfolge auf zu ziehen. Dabei bekommt eine Spielerin volle Sicht, oder eingeschränkte Sicht auf das aktuelle Spielbrett, abhängig von den Einstellungen der Spielerin. Der Server wartet nach dem aufrufen von `GetMove()` ab, bis die Spielerin mitteilt welchen Zug sie ausführen möchte. Der Server führt diesen Zug nach Überprüfung aus. Daraufhin wird überprüft ob das Spiel vorbei ist. Dies wird wiederholt bis Spielende. Sollte dieses eintreten, dann gibt der Server den Spielausgang, Zugfolge, Anzahl der Rollouts für jeden Zug zurück.

4.1.3 KIs

Die KIs implementieren die Funktion `GetMove()`. `GetMove()` generiert einen Spielzug, der an den Spielserver zurück gegeben wird. Außerdem teilen die KIs dem Server mit, welche Sicht sie auf das Spielbrett erhalten.

Die POMCP-KI bekommt bei ihrer Initialisierung Einstellungen übergeben. Innerhalb dieser Einstellungen können dann Hybride übergeben werden welche dann in den Simulationen von POMCP zum Einsatz kommen. Die genutzten Einstellungen werden zusammen mit Sieg/Niederlage und Namen der KIs in CSV Dateien für die Auswertung gespeichert.

4.2 Bitboards

Eine effektive Möglichkeit Spielzüge zu generieren ist mithilfe von Bitboards. Ein Bitboard ist dabei eine 64-Bit Zahl. Jede Eins repräsentiert in dieser Zahl eine Schachfigur auf einem Feld. Dabei stellt das Least Significant Bit (LSB) das Feld A1 dar und das

8	56	57	58	59	60	61	62	63
7	48	49	50	51	52	53	54	55
6	40	41	42	43	44	45	46	47
5	32	33	34	35	36	37	38	39
4	24	25	26	27	28	29	30	31
3	16	17	18	19	20	21	22	23
2	8	9	10	11	12	13	14	15
1	0	1	2	3	4	5	6	7
	a	b	c	d	e	f	g	h

Abbildung 4.2: Bitboard-Repräsentation eines Schachbretts. Quelle: In Anlehnung an [Ras04]

Most Significant Bit H8. Dabei werden die Bits dazwischen beginnend vom LSB reihenweise mit steigender Signifikanz auf die Schachfelder abgebildet 4.2. Da jedes Bit eines Bitboards nur zwei Zustände darstellen kann werden mehrere Bitboards benötigt um ein Spielfeld vollständig abzubilden. Für jeden Figurentyp (Bauer, Turm, Springer, Läufer, Dame, König) jeder Farbe (Weiß/Schwarz) wird ein Bitboard benötigt. Mögliche Züge werden bei der Spielinitialisierung für jede Position und Figurentyp (bei den Bauern auch pro Farbe) berechnet. Die vorberechneten möglichen Züge werden mit den Bitboards der Figuren mit logischen Operationen verknüpft um Züge zu generieren. Dabei wird darauf geachtet, dass Züge ausgeschlossen werden, bei denen auf eigene Figuren gezogen wird.

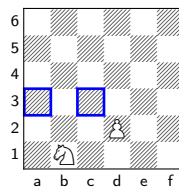


Abbildung 4.3: Mögliche Züge für den Springer auf B1 blau markiert, der Springer kann nicht auf D2 ziehen da das Feld durch eine eigene Figur blockiert wird. Quelle: Eigene Darstellung

$$\begin{aligned}
K(1) &= \text{SPRINGER_ZÜGE}[1] \text{ UND NICHT WEIßES_BESETZUNG_BITBOARD} \\
\text{SPRINGER_ZÜGE}[1] &= 10100100000000 \\
\text{WEIßES_BESETZUNG_BITBOARD} &= 00001000000010 \\
\text{NICHT_WEIßES_BESETZUNG_BITBOARD} &= 11110111111101 \\
K(1) &= 10100000000000
\end{aligned} \tag{4.1}$$

Für die in Abb. 4.3 dargestellte Situation wird in Gleichung (4.1) die Zuggenerierung veranschaulicht. Dabei ist $K(1)$ die Funktion für die Zuggenerierung von Springern, aufgerufen mit Parameter 1 da der Springer auf B1 steht. In dem WEIßES_BESETZUNG_BITBOARD sind alle weißen Bitboards verodert. Für die Figuren Turm, Läufer und Dame ist die Zuggenerierung komplexer. Diese Figuren können in Ihre Laufrichtungen gezogen werden, bis zu der ersten blockenden Figur oder dem Ende des Spielfeldes. Für diese Figuren iterieren wir entlang Ihrer Bewegungsrichtungen, bis wir auf eine blockende Figur stoßen.

4.3 Statische Speicherallokation

Eine frühere Implementierung hatte Performance Probleme durch eine hohe Anzahl dynamischer Speicherallokationen. Daraufhin wurden dynamische Datenstrukturen innerhalb von POMCP durch statische ersetzt. So wurde der Spielbaum von POMCP als Ringpuffer implementiert. Solange im Ringpuffer freie Knoten zur Verfügung stehen kann der Spielbaum expandieren. Ist dies nicht der Fall, dann werden Rollouts durchgeführt ohne dabei den Spielbaum zu expandieren. Infolgedessen besitzt nicht jeder einzelne Knoten einen Partikelfilter wie in Standard POMCP, sondern Partikelfilter werden nur am Wurzelknoten gespeichert. Und müssen nach jedem Zug erst generiert werden.

4.4 Partikelfilter

Nach jedem Zug wird der Partikelfilter aktualisiert. Dabei wird eine festgelegte Anzahl (normalerweise 10.000) zufälliger Partikel aus dem alten Partikelfilter gezogen und diese zur Bildung des neuen Partikelfilters genutzt. Das bedeutet, dass der Spielzustand aus dem Partikelfilter gemäß der letzten Aktion der Spielerin aktualisiert wird und zusätzlich eine zufällige gegnerische Aktion ausgeführt wird.

Von dem so entstandenen Zustand wird der ViewHash-Wert berechnet. Entspricht dieser Hashwert dem Hashwert, den die aktuelle Sicht mit Berechnung des ZobristHash-Wertes hat, ist der Spielzustand mit der aktuellen Sicht kompatibel, und wir nehmen ihn in den Partikelfilter auf.

Sollte der Partikelfilter nach der Aktualisierung des alten Partikelfilters leer sein (d. h. keine Spielzustände waren mit der Sicht kompatibel), dann müssen neue erzeugt werden, da ansonsten POMCP keine Annahmen mehr über mögliche gegnerische Positionen hat.

Um neue Partikel für den Partikelfilter zu generieren, gehen wir wie folgt vor:

Wir nutzen zunächst den alten Partikelfilter zur Generierung neuer Partikel, wie oben beschrieben. Wenn diese nicht kompatibel sind mit der Sicht der Spielerin, wird der Spielzustand gemäß der Sicht der Spielerin korrigiert. Das bedeutet: Steht beispielsweise an dem vorgeschlagenen Partikel auf einem Feld eine Dame, wir sehen aber am Spiel, dass das Feld leer ist, dann löschen wir die Dame.

Es kommt also offensichtlich zu Informationsverlusten, trotzdem bekommen wir durch den aktualisierten Partikelfilter gute Annahmen über den Spielzustand.

4.5 Bewertungsfunktionen

Es kommen zwei Bewertungsfunktionen innerhalb dieser Arbeit zum Einsatz: eine simple Bewertungsfunktion für das Spiel Dark Pawn Chess und in den beiden komplexeren Spielen eine von Shannon vorgeschlagene Bewertungsfunktion, die für diese Arbeit modifiziert worden ist.

4.5.1 Dark Pawn Chess

Bei Dark Pawn Chess nutzen wir H als Heuristik.

$$H_P = \frac{P - P' + C - C'}{10} \quad (4.2)$$

Dabei steht P für die Anzahl der Bauern und C für die Anzahl der gedeckten Bauern. Wobei P die derzeitig evaluierte Spielerin ist und P' die gegnerische Spielerin. Nach dem addieren der Materialdifferenz und der Deckungsdifferenz dividieren wir durch 10, um H_P auf $[-1,1]$ zu normieren. Denn die maximale Materialdifferenz beträgt 5 sowie die maximale Deckungsdifferenz beträgt auch 5.

4.5.2 Dark Los Alamos Chess und Dark Chess

Als Heuristik für Dark Los Alamos Chess verwenden wir eine von Shannon[Sha50] eingeführte Bewertungsfunktion:

$$S = 200(K - K') + 9(Q - Q') + 5(R - R') + 3(N - N') + (P - P') - 0.5(D - D' + I - I') + 0.1(M - M') \quad (4.3)$$

Dabei steht K für den König, Q für die Dame, R für die Türme, N für die Springer, P für die Bauern. D sind gedoppelte, und I sind einzelne Bauern. Und M beschreibt die Mobilität einer Spielerin, also die Anzahl der möglichen Züge.

Innerhalb dieses Projekt wurde die Shannon Bewertungsfunktion angepasst. Da POM-CP Spiele mit $\{1, 0, -1\}$ bewertet, normieren wir die Bewertungsfunktion von Shannon auf das Intervall $[-1, 1]$. Außerdem wurde die Bewertung des Königs entfernt der die Bewertung dominiert, stattdessen wird überprüft ob das Spiel zu Ende ist. Im Falle eines

Spielende liefert die Bewertungsfunktion $\{1, -1\}$ zurück. Die Bewertungsfunktion H_C wird mit dem Spielzustand s und der Spielerin p aufgerufen.

$$H_C(s, p) = \begin{cases} 1, & s \text{ ein Gewinnzustand für Spielerin } p \text{ ist,} \\ -1, & s \text{ ein Verlustzustand für Spielerin } p \text{ ist,} \\ H_L(s), & \text{sonst} \end{cases}$$

$$H_L(s) = \max \left(\min \left(\frac{S_D(s)}{M_D(s)}, 1 \right), -1 \right) \quad (4.4)$$

$$S_D(s) = 9(Q - Q') + 5(R - R') + 3(N - N') + (P - P') - 0.5(D - D' + I - I') + 0.1(M - M') \quad (4.5)$$

$$M_D(s) = \frac{9(Q + Q') + 5(R + R') + 3(N + N') + (P + P')}{2} \quad (4.6)$$

Für Dark Chess erweitern wir H_C um die fehlenden Bauern und Läufer.

4.6 Gegnermodellierung

Gegnermodellierung beschreibt bei der Suche nach der besten Aktion das Verhalten, d. h. die Strategie, die wir für den Gegner bei unseren Berechnungen annehmen. Bei Minimax ist diese Modellierung zum Beispiel sehr einfach und implizit: Der Gegner wird so eingeschätzt, dass er das für sich beste Ergebnis wählt.

POMCP hat jedoch oft eine Gegnermodellierung, bei der die Züge des Gegners im Suchbaum zufällig gleichverteilt ausgewählt werden.

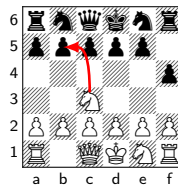


Abbildung 4.4: Weiß am Zug und untersucht die Aktion C3-B5. Quelle: Eigene Darstellung

Das bedeutet: Angenommen, die weiße Spielerin befindet sich in Spielsituation 4.4. Führt Weiß den Zug C3-B5 aus, und wir simulieren mögliche Antworten auf diesen Zug so, dass Schwarz zufällig gleichverteilt einen Zug wählt, dann wird Schwarz nur in 1/10

der Fälle den Springer schlagen, d. h. in 9/10 Simulationen kann Weiß im nächsten Zug das Spiel mit B5-D6 gewinnen.

Der mögliche Verlust eines Springers im Tausch gegen die hohe Wahrscheinlichkeit, das Spiel zu gewinnen, lohnt sich aus der Sicht der weißen Spielerin.

Um diesen Trugschluss zu verhindern, nutzen wir für die Gegnermodellierung eine Greedy-Zugauswahl, wobei manchmal (mit einer gewissen Wahrscheinlichkeit) vom besten Zug abgewichen wird und ein zufälliger Zug gewählt wird.

5 Experimente

Um POMCP-Hybride auszuwerten wird zuerst eine POMCP-Baseline bestimmt. Dafür wird die UCT-Konstante optimiert. Für jede Kombination aus KI und zu optimierendem Parameter werden 100 Spiele durchgeführt. Dabei werden 50 Spiele als weiße Spielerin und 50 als schwarze durchgeführt. Die KIs bekommen pro Zug 1 Sekunde Zeit. Als wichtigste Metrik verwenden wir die Siegesrate der Spielerinnen. Dabei gilt: $\text{Siegesrate} = (\text{Siege} + 0,5 * \text{Unentschieden}) / \text{gesamte Spiele}$.

Getestet wird mit einer AMD Ryzen 5 5625U CPU und 16GB Arbeitsspeicher. Die folgenden Diagramme sind Fehlerbalkendiagramme, diese zeigen neben der Siegesrate auch das 95-% Konfidenzintervall an. Nach 100 Spielen werden vielversprechende Hybride ausgewählt und 400 weitere Spiele simuliert, um die Unsicherheit im Ergebnis zu minimieren.

5.1 Dark Pawn Chess

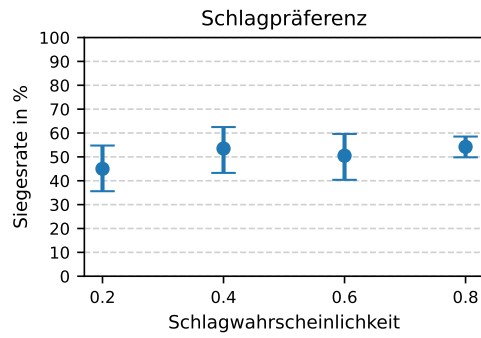
Für eine Baseline KI in Dark Pawn Chess wird zuerst die UCT-Konstante c optimiert. Dabei wurde $c \in \{0, 1; 0, 5; 1; 5; 10\}$ gewählt, während die Gegenspielerin mit einem festen $c = 1$ spielte. Nach 100 Spielen lag die höchste Siegesrate bei $c = 5$, mit 52,5%. Dabei liegen die Siegesraten von 1,5,10 aber sehr nah beieinander mit einem Unterschied von 1%. Die Siegesrate von 0,1 und 0,5 ist dagegen mit kleiner 43% deutlich geringer.

Bei den Ergebnissen fällt die Diskrepanz der Siegesrate zwischen den Spielfarben auf. Mit $c = 5$, konnte die Spielerin 34% ihrer Spiele mit Weiß gewinnen und 71% der Spiele mit Schwarz. Dies könnte durch den geltenden Zugzwang bedeuten, dass Weiß zuerst seine Deckung aufgeben muss und Schwarz dadurch bevorteilt ist.

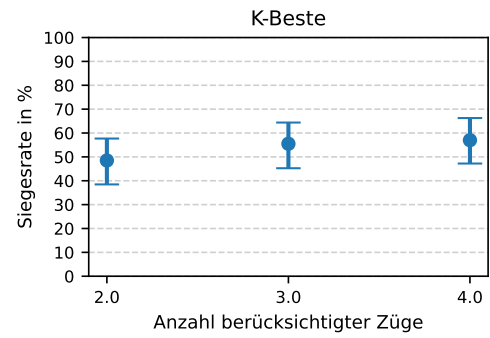
Als Baseline wird POMCP mit der UCT Konstanten $c=5$ genutzt. Der Schlagpräferenz-Hybrid mit Schlagwahrscheinlichkeit 0,8 lag nach 100 Simulationen bei über 60% Siegesrate. Um zu erkennen, ob das Ergebnis signifikant ist wurde diese Einstellung vom Hybrid weitere 400-mal simuliert. Nach 500 Spielen wies der Hybrid eine Siegesrate von 54,2% auf (siehe Abb. 5.1 (a)), ist damit aber noch nicht signifikant besser als die Baseline.

Der K-Beste Hybrid funktioniert am besten in Dark Pawn Chess, je höher k ist. Dabei zeigte sich $k = 4$ am erfolgreichsten, mit einer Siegesrate von 57%. D.h. der k -Beste-Hybrid beschneidet in Dark Pawn Chess nicht stark die möglichen Züge, aber die gewichtete Zugauswahl steigert die Siegesrate (siehe Abb. 5.1 (b)).

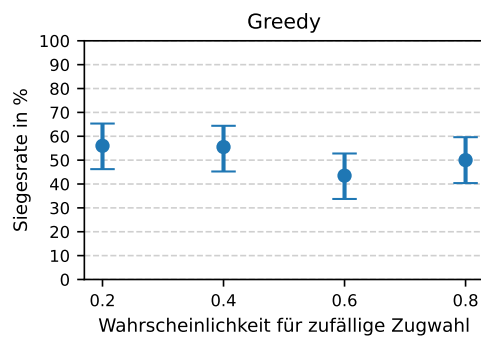
Der Greedy-Hybrid ist am erfolgreichsten, bei kleinem Epsilon also einer geringen



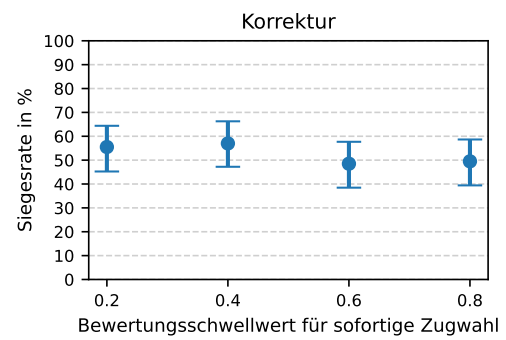
(a)



(b)



(c)



(d)

Abbildung 5.1: Siegesraten des Schlagpräferenz-Hybrid (a), des K-Beste-Hybrid (b), Greedy-Hybrid (c) und Korrektur-Hybrid (d), unter Variation ihrer Parameter. Quelle: Eigene Darstellung

Anzahl an Zufallszügen. Bei Epsilon=2, liegt die Siegesrate bei 56% und ist damit am höchsten (siehe Abb. 5.1 (c)).

Am erfolgreichsten ist der Korrektur-Hybrid, wenn schon Aktionen, die kleinen Materialvorteil bringen, im Rollout bevorzugt ausgewählt werden. So siegt der Hybrid 57% der Spiele, wenn Züge im Rollout Greedy gewählt werden, welche eine Bewertung von >0.4 haben (vergleiche Abb. 5.1 (d)).

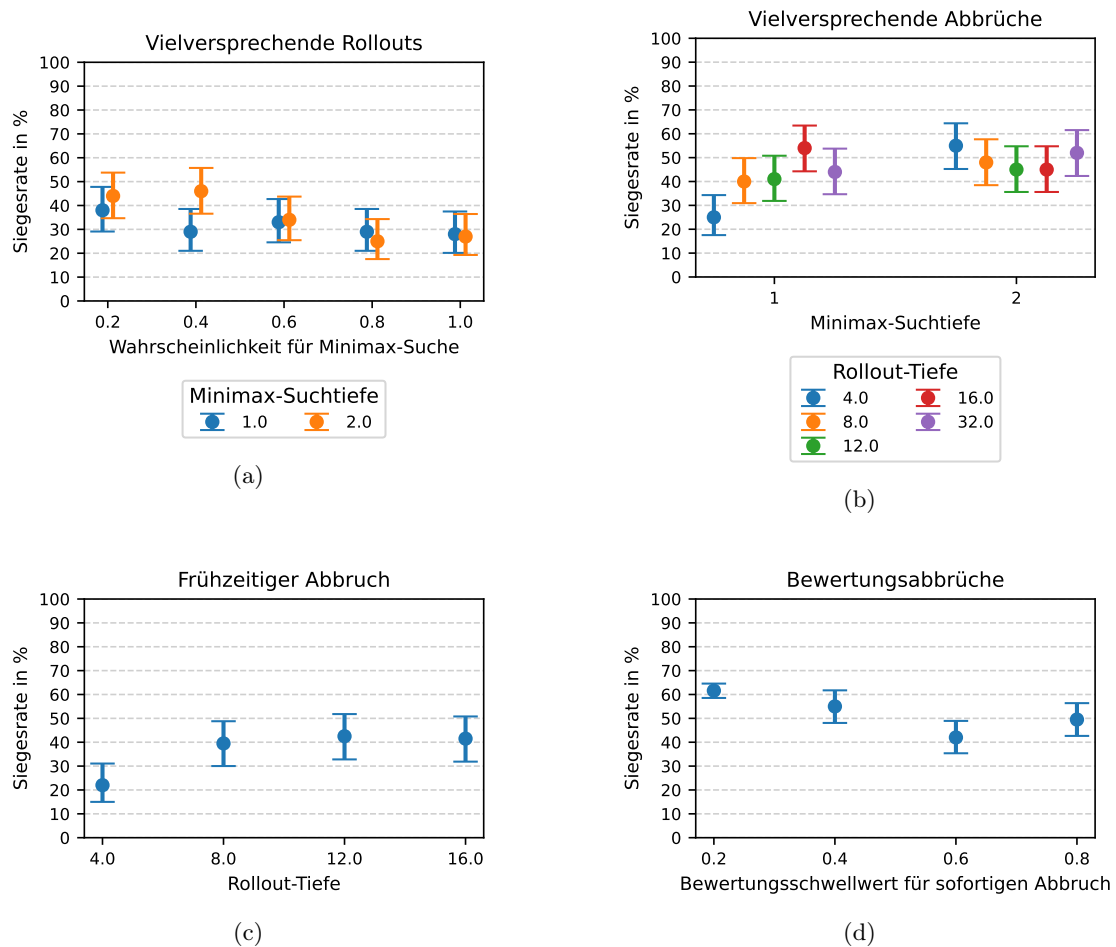


Abbildung 5.2: Siegesraten des Vierversprechende-Rollouts (a), des Vierversprechende-Abbrüche-Hybrid (b), Frühzeitige-Abbrüche-Hybrid (c) und Bewertungs-Abbruch-Hybrid (d), unter Variation ihrer Parameter. Quelle: Eigene Darstellung

Vierversprechende-Rollouts erwies sich in Dark Pawn Chess als wenig erfolgreich. So erzielte die beste getestete Variante 46% Siegesrate mit einer Minimax-Suchtiefe 2 und einer Wahrscheinlichkeit von 0.4 die Minimax-Suche auszuführen (siehe Abb. 5.2 (a)).

Der Vielversprechende-Abbrüche-Hybrid hängt von zwei Parametern ab. Als beste Parameterisierung erwies sich Abbruchtiefe 4 und Minimax Suchtiefe 2, dabei konnte der Hybrid 55% der Spiele gewinnen (siehe Abb. 5.2 (b)). Auch die Parameter Kombination von Minimax-Suchtiefe 1 und Rollout-Tiefe 16 erreichte 54% Siegesrate.

Die Siegesrate des Frühzeitige-Abbrüche-Hybrid steigt mit der Abbruchtiefe an. Dabei liegt die Siegesrate bei Abbruchtiefen 12, 16 dicht aneinander mit 42,5% und 41,5% (siehe Abb. 5.2 (c)).

Der Bewertungs-Abbrüche-Hybrid weist bei einem Abbruchschwellewert von 0,2%, ab dem ein Spiel also Gewonnen/Verloren gewertet wird, eine Siegesrate von 61,6% in 500 Spielen (siehe Abb. 5.2 (d)).

5.2 Dark Los Alamos Chess

Um die Baseline zu bestimmen wurden die UCT Konstante c optimiert und die Gegnermodellierung. Experimentell zeigte sich $c = 5$ am stärksten, dabei wurde mit $c \in \{0, 1; 0, 5; 1; 5; 10\}$ getestet. Bei der Gegnermodellierung erwies sich ein Wert von 0,4 am erfolgreichsten. Die Gegnermodellierung wählt also in 40% der Fälle einen zufälligen Zug, sonst den bestbewertesten.

Der Schlagpräferenz-Hybrid zeigte sich am stärksten bei einer 40%-Chance einen Schlagzug zu wählen. Dabei erzielte der Hybrid eine Siegesrate von 57% in 1000 Spielen (siehe Abb. 5.3 (a)).

K-Beste zeigte sich bei dem beschneiden der Zugauswahl auf die vier besten Züge am erfolgreichsten (siehe Abb. 5.3 (b)). Dabei erzielte der Hybrid eine Siegesrate von 57%.

Der Greedy-Hybrid ist am erfolgreichsten mit einer Wahrscheinlichkeit von 0,6, dass eine zufällige Aktion gewählt wird und ansonsten die bestbewerteste (siehe Abb. 5.3 (c)). Die beste Greedy Variante erreicht eine Siegesrate von 57%.

Der Korrektur-Hybrid ist am erfolgreichsten wenn die erste Aktion welche in einen Spielzustand mit einer Bewertung $>0,8$ ausgewählt wird. Mit dieser Einstellung erreicht der Korrektur-Hybrid eine Siegesrate von 48% (siehe Abb. 5.3 (d)).

Der Vielversprechende-Rollouts-Hybrid hat die höchste Siegesrate von 60% bei einer Wahrscheinlichkeit von 0.2 eine Minimax-Suche mit Suchtiefe 1 auszuführen um einen Zug im Rollout zu wählen (siehe Abb. 5.4 (a)). Bei Minimax-Suchtiefe 2 ist es am erfolgreichsten jeden Zug mit Minimax zu ermitteln(Wahrscheinlichkeit=1).

Vielversprechende Abbrüche zeigten sich bei einer Minimax-Suchtiefe von 1, am besten. Mit einer Rollout-Tiefe 20 gewann der Hybrid 58% seiner Spiele (siehe Abb. 5.4 (b)). Bei

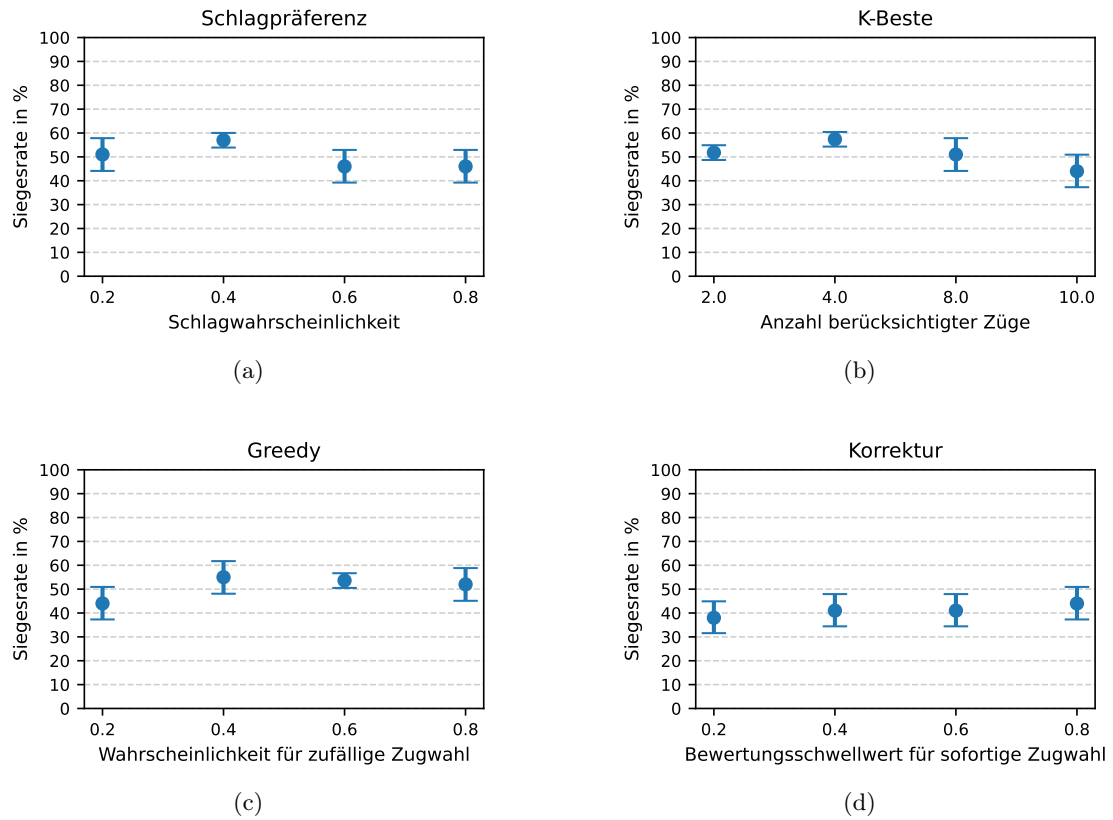


Abbildung 5.3: Siegesraten des Vielversprechende-Rollouts (a), des Vielversprechende-Abbrüche-Hybrid (b), Frühzeitige-Abbrüche-Hybrid (c) und Bewertungs-Abbruch-Hybrid (d), unter Variation ihrer Parameter. Quelle: Eigene Darstellung

Minimax-Suchtiefe 2 und Rollout-Tiefen 6,20 konnte der Hybrid 57% seiner Spiele gewinnen. Die deutlich aufwendigere Minimax-Suchtiefe 4 ist in den Siegesraten abgeschlagen. Die Rollout-Tiefe 40 hat eine niedrigere Siegesrate als die kürzeren Rollouts.

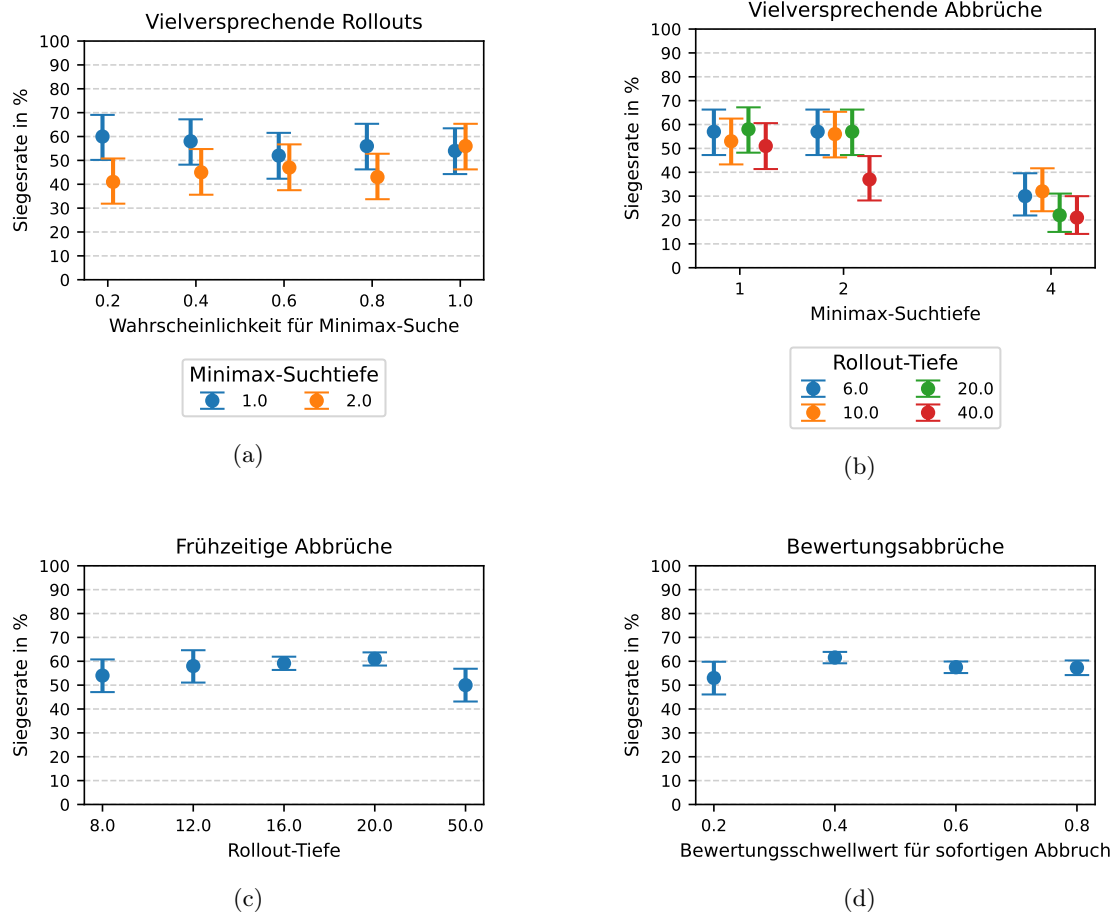


Abbildung 5.4: Siegesraten des Vielversprechende-Rollouts (a), des Vielversprechende-Abbrüche-Hybrid (b), Frühzeitige-Abbrüche-Hybrid (c) und Bewertungs-Abbruch-Hybrid (d), unter Variation ihrer Parameter. Quelle: Eigene Darstellung

Der Informierte-Abbrüche-Hybrid blieb in allen getesteten Einstellungen unter einer Siegesrate von 40%.

Die Siegesrate von Frühzeitiger-Abbruch ist mit 61% am höchsten bei einer Abbruchtiefe von 20 (siehe Abb. 5.4 (c)). Bei Abbruchtiefe 50 liegt eine Siegesrate von 50% vor, was vermuten lässt, dass entweder die meisten Rollouts in Los Alamos Chess kürzer sind als Tiefe 50 oder die Dauer der Bewertungsfunktion zeitlich ähnlich lang ist wie tiefere Rollouts zu beenden.

Der Bewertungs-Abbruch-Hybrid erreicht bei einem Abbruchschwellwert von 0.4 eine maximale Siegesrate von 61,6% (siehe Abb. 5.4 (d)). Für höherer Abbruchschwellwerte bleiben die Siegesraten leicht darunter. Ein Abbruch schon bei wenig Materialvorteil wirkt sich aber nur leicht positiv auf die Siegesrate aus.

5.3 Dark Chess

In Dark Chess erwies sich die UCT-Konstante $c = 0,1$ am erfolgreichsten ($n=100$). Wobei ein klarer Trend erkennbar ist, denn nur $c < 1$ hat Siegesraten größer 50%. Die Greedy-Gegnermodellierung zeigt sich bei einer Wahrscheinlichkeit von 0,2 einen Zufallszug auszuführen am erfolgreichsten mit einer Siegesrate von 64,5%. Diese Werte für c und die Gegnermodellierung nutzen wir für die Baseline. Im folgenden werden die Ergebnisse der Hybride diskutiert.

Bei dem Schlagpräferenz-Hybriden führte eine hohe Wahrscheinlichkeit im Rollout Schlagzüge auszuführen zu einer ausgeglichenen Siegesrate. Bei niedrigeren Schlagwahrscheinlichkeiten fallen die Siegesraten leicht ab. Am erfolgreichsten erwies sich eine Schlagpräferenz von 0,8 mit einer Siegesrate von 50% (siehe Abb. 5.5 (a)).

Der Hybrid K-Beste ist am erfolgreichsten mit $k = 24$ mit einer Siegesrate von 62,5%. Es lohnt sich also die Zugauswahl in Dark Chess nicht zu stark zu beschränken und die Zugauswahl gewichtet nach der Bewertung durchzuführen (siehe Abb. 5.5 (b)).

Der Greedy-Hybrid hat eine Siegesrate von 57%, bei einem Epsilon von 0,4 (siehe Abb. 5.5 (c)).

Mit einer Siegesrate von 66,5% ist der Korrektur-Hybrid bei einem Schwellwert von 0,2 am erfolgreichsten. Dabei fällt auf nur bei Schwellwert 0,6 ist das untere Ende des 95-Konfidenzintervall unter der 50% Siegesrate (siehe Abb. 5.5 (d)). Bei hohem Schwellwert findet der Korrektur-Hybrid nur selten Spielzustände die diesen übertreffen. Deshalb ähnelt sein Verhalten jenem des K-Besten-Hybriden mit hohem k -Wert.

Aufgrund der langen Simulationszeit für Dark Chess Spiele setzen wir den Parameter für die Minimax-Suchtiefe auf 2. Dabei zeigten sich Vielversprechende-Rollouts sehr erfolgreich gegen die Baseline. So erreicht der Hybrid bei einem Abbruchschwellwert von 0,6 eine Siegesrate von 72%. Aber auch die Schwellwerte 0,7 und 0,8 zeigten sich erfolgreich mit Siegesraten von über 60%. Die anderen Parameter endeten auch in einer Siegesrate von 60% (siehe Abb. 5.6 (a)).

Auch bei dem Vielversprechende Abbrüche Hybrid testen wir nur einen Parameter und fixieren eine Minimax-Suchtiefe von 2. Dabei konnte bei einer Abbruchtiefe 2, 6 eine Siegesrate von 74% erzielt werden (vergleiche Abb. 5.6 (b)). Bei späteren Abbrüchen fällt die Siegeswahrscheinlichkeit schon bei Tiefe 20 auf 60,5% ab, ab Tiefe 30 pendelt sich die

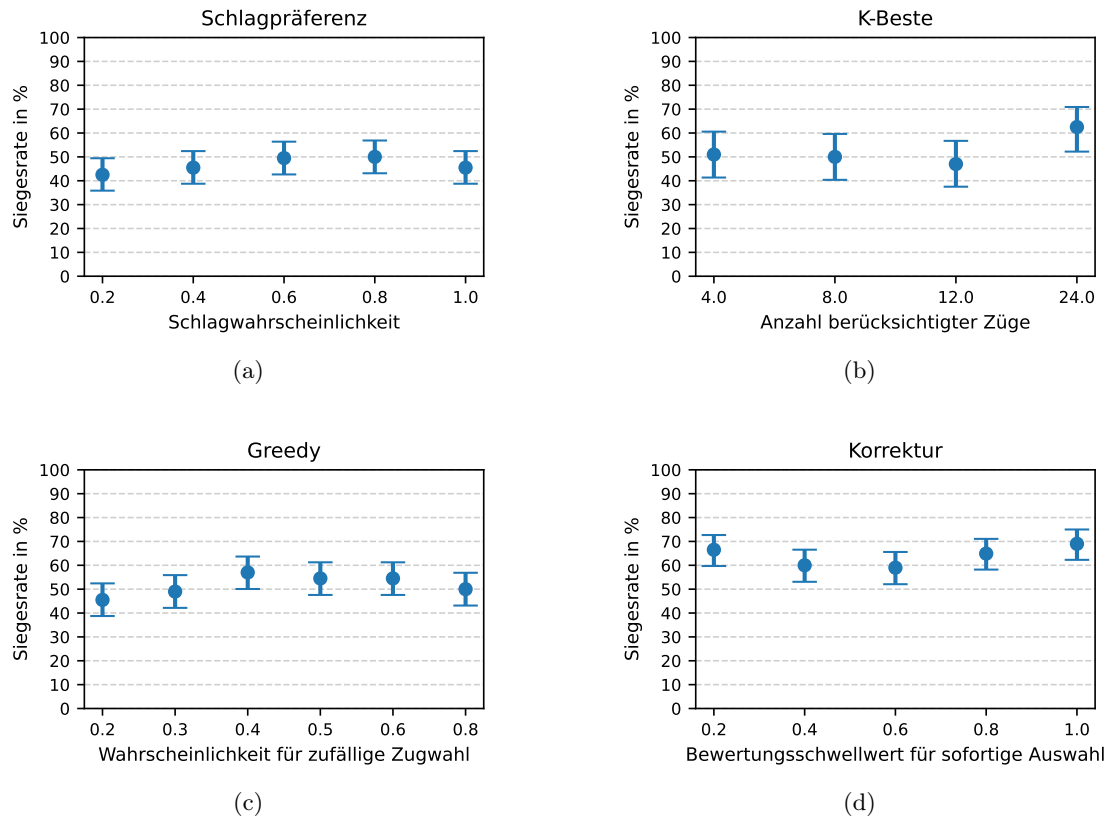


Abbildung 5.5: Siegesraten des Vielversprechende-Rollouts (a), des Vielversprechende-Abbrüche-Hybrid (b), Frühzeitige-Abbrüche-Hybrid (c) und Bewertungs-Abbruch-Hybrid (d), unter Variation ihrer Parameter. Quelle: Eigene Darstellung

Siegesrate bei 50% ein. Auch bei dem Frühzeitigen-Abbruch-Hybrid setzt sich dies fort.

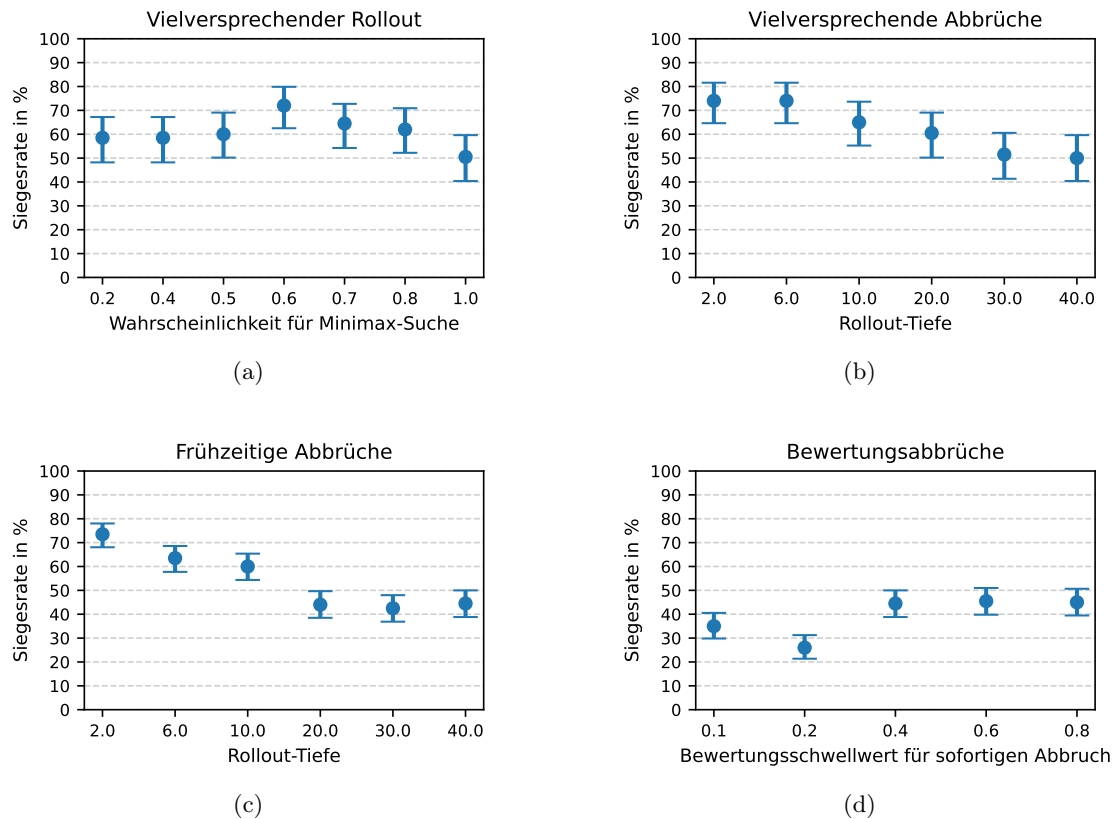


Abbildung 5.6: Siegesraten des Vielversprechende-Rollouts (a), des Vielversprechende-Abbrüche-Hybrid (b), Frühzeitige-Abbrüche-Hybrid (c) und Bewertungs-Abbruch-Hybrid (d), unter Variation ihrer Parameter. Quelle: Eigene Darstellung

So zeigte sich dieser am erfolgreichsten mit 76% Siegesrate bei einer Abbruchtiefe von 2. Bei späten Abbrüchen fällt die Siegesrate auf unter 50% (siehe Abb. 5.6 (c)). Es fällt auf wie wichtig es ist in Dark Chess unter der bestehenden Unsicherheit über den aktuellen Zustand kurzfristige Erfolge anzustreben.

Der Bewertungs-Abbruch-Hybrid hat bei einem Abbruchschwellwert von 0.6 die höchste Siegesrate von 45,5 % (siehe Abb. 5.6 (d)). Ein Abbruch der Suche bei einem kleinen Vorteil wird nicht belohnt, so liegt die Siegesrate von dem Schwellwert 0,2 bei nur 26%. Denn obwohl 0,2 schon ein deutlicher Vorteil ist in Dark Chess, ist es zu früh um das Spiel als Gewonnen/Verloren anzusehen.

5.4 Analyse der Hybride in den verschiedenen Spielen

In allen Spielen konnten Hybride eine signifikante Siegesrate gegenüber der Baseline vorweisen. In Dark Pawn Chess(DPC) konnte der Bewertungs Abbrüche Hybride eine 61,6% Siegesrate erzielen. In Dark Los Alamos Chess(DLAC) konnten Vielversprechende Abbrüche, Frühzeitiger-Abbruch und Bewertungs Abbrüche eine Siegesrate von über 60% aufweisen. In Dark Chess(DC) konnten fünf Hybride eine Siegesrate von über 60% aufweisen, davon drei sogar über 70% (siehe Tabelle 5.1).

Der Schlagpräferenz Hybrid konnte in allen drei Spielen nicht erfolgreich eingesetzt werden. K-Beste war in allen Spielen erfolgreich, wobei in Dark Pawn Chess und Dark Chess es am erfolgreichsten war bei hohem k , während es bei Dark Los Alamos Chess gut funktionierte k klein zu wählen. DPC und DC profitieren beide von einer zufällig gewichteten Zugwahl, während dies nicht der Fall bei DLAC ist. Der Korrektur Hybrid unterstützt diese Theorie, wenn kein Zug über dem Schwellwert gefunden wird, dann erfolgt die Zugwahl auch zufällig gewichtet. Dabei schneidet der Hybrid wieder gut in DPC und DC ab, während der Hybrid nicht gut in DLAC abschneidet. Der Greedy Hybrid erzielte in allen Spielen Siegesraten von über 55%. Vielversprechende Rollouts, Vielversprechende Abbrüche und Frühzeitiger-Abbruch konnten alle mit ansteigender Komplexität des Spiels ihre Siegesraten deutlich erhöhen. Bewertungs Abbrüche konnte nur in DPC und DLAC erfolgreich eingesetzt werden, während in DC die Siegesrate auf unter 50% einbricht.

DPC ist das Spiel mit der niedrigsten Komplexität. Die Baseline konnte hier eine sehr

Hybrid	DPC-Siege(%)	DLAC-Siege(%)	DC-Siege(%)
Schlagpräferenz	50	57	50
k-beste	57	57,4	62,5
Greedy	56	55	57
Korrektur	57	44	69
Vielversprechende Rollouts	46	60	72
Vielversprechende Abbrüche	55	58	74
Frühzeitiger-Abbruch	42,5	61	73,5
Bewertungs Abbrüche	61,6	61,6	45

Tabelle 5.1: Die Siegesraten der optimierten Hybride in den drei getesteten Spielen

hohe Anzahl an Rollouts durchführen. Dadurch ist wenig steigerungspotential übrig, um in diesem Spiel die Spielstärke der KI anzuheben.

In DLAC steigt die Komplexität des Spiels stark an. So werden pro Sekunde nicht mehr mehr als 50.000 Rollouts/s durchgeführt sondern nur noch mehr als 2600/s (siehe Tabelle 5.2). Zufällige Rollouts leiden unter der fehlenden Taktik um Spiele zu beenden. Dadurch steigt hier der Erfolg der Hybride an.

In DC ist die Komplexität des Spieles sehr hoch. Da die KI hier die Probleme wie in DLAC hat, können Hybride in diesem Spiel deutlich ihre Siegesrate gegenüber der Baseline ausbauen. Dabei zeigte sich kurzfristiges Planen sehr erfolgreich. So ist die Rollout-Tiefe 2 in Vielversprechende Abbrüche und Frühzeitige Abbrüche am erfolgreichsten.

Hybrid	DPC-Rollouts/s	DLAC-Rollouts/s	DC-Rollouts/s
Schlagpräferenz	51296	3242	1735
k-beste	50945	2631	1000
Greedy	60410	3412	1339
Korrektur	58596	3143	1301
Vielversprechende Rollouts	93578	4911	1343
Vielversprechende Abbrüche	105635	8774	1497
Frühzeitiger-Abbruch	51425	3503	1577
Bewertungs Abbrüche	51994	3280	1453

Tabelle 5.2: Der Median der Rollouts/s der optimierten Hybride in den drei getesteten Spielen

6 Fazit

6.1 Zusammenfassung

In dieser Arbeit wurden drei Schachspiele mit unvollständigen Informationen umgesetzt. Dark Pawn Chess ein Schachspiel auf einem 5×5 Feld bestehend nur aus Bauern, welche die gegnerische Grundline erreichen müssen um zu gewinnen. Dark Los Alamos Chess ist ein 6×6 Schachspiel, dabei fehlt gegenüber dem herkömmlichen Schach die Läufer. Dark Chess spielt auf einem 8×8 Schachbrett und wird fast mit identischen Regeln wie Schach mit unvollständigen Informationen gespielt. Sichtbar sind nur eigene Figuren, oder Felder auf denen ihre Figuren ziehen könnten. Partially Observable Monte Carlo Planning, eine Anpassung des Monte Carlo Tree Search Algorithmus, wurde dabei auf die implementierten Schachspiele angewandt. Im Anschluss daran wurden Hybride getestet, welche entweder die Qualität der Rollouts steigern sollte oder die Rollout Anzahl erhöhen sollte.

Während in Dark Pawn Chess nur ein Hybrid eine signifikante Steigerung im Vergleich zur Baseline darstellte, waren es in Dark Los Alamos Chess drei und in Dark Chess fünf Hybride. Je komplexer das Spiel ist, desto mehr steigt die Unsicherheit über den tatsächlichen Spielzustand an. Während in Dark Pawn Chess oft der genaue Spielzustand bekannt ist, oder nur wenige dafür infrage kommen und Rollouts schnell durchgeführt werden können, sind Hybride nur wenig effektiv. So hatte der beste Hybrid Bewertungs Abbrüche eine Siegesrate von 61,6%. In Dark Los Alamos Chess steigt die Anzahl der die möglichen Spielzustände stark an, aber auch der Aufwand um mögliche Züge und die Bewertungsfunktion zu berechnen. Dadurch sinkt die Anzahl der durchgeführten Rollouts stark ab. Interessanterweise konnte in Dark Los Alamos Chess kein Hybrid den besten von Dark Pawn Chess übertreffen. Denn auch in Dark Los Alamos Chess hatte der Bewertungs Abbrüche Hybrid mit 61,6% die höchste Siegesrate. Durch den Einsatz einer Bewertungsfunktion bekommt die KI eine Taktik. Diese macht sich dann vor allem in Dark Chess bemerkbar. Hier konnte der Vielversprechende Abbrüche Hybride durch sehr frühe Abbrüche (Rollout-Tiefe 2 und 6) eine Siegesrate von 74% erzielen. Durch die hohe Unsicherheit wird es wichtig eine kurzfristige Strategie zu entwickeln.

6.2 Probleme

Eine erste Implementierung des Projektes mit dynamischer Speicherallokation (Listen, Maps), ermöglichte nur wenige Rollouts/s. Die Implementierung von dem komplexeren Spiel Dark Los Alamos Chess verkomplizierte dies. Daher wurde entschieden, statische Speicherallokation anzuwenden. Dies machte einen Knotenringpuffer, Partikelverwaltung

im Wurzelknoten notwendig.

Außerdem erwies es sich als schwierig eine gute Auswahl an zu testender Parameter zu finden, durch die große Anzahl an getesteten Hybriden und Spielen. Das Problem wurde gelöst, indem pro Hybride nur wenige Parameter getestet wurden und später die vielversprechendsten davon um weitere Simulationen ergänzt wurden.

6.3 Ausblick

In dieser Arbeit wurde die Gegnermodellierung durch ϵ -Greedy vereinfacht dargestellt. Um diese zu verbessern wäre es möglich explizite Gegnerknoten einzuführen. Die Knoten im Suchbaum würden dann Minimax-ähnlich die eigene Sicht maximieren und aus der gegnerischen Sicht minimieren. Auch müsste ein Partikelfilter aus gegnerischer Sicht eingeführt werden. Denn ansonsten gehen wir davon aus, dass der Gegner perfekte Sicht hat auf unsere Figuren, was aber nur selten der Wahrheit entspricht.

Wenn sich im Partikelfilter keine Partikel mehr befinden, müssen welche neugeneriert werden. In dieser Arbeit wurde dies gelöst in dem die Beobachtung mit zufälligen alten Partikeln zusammengeführt wurden. Das Problem dabei ist, dass dabei Informationen verloren gehen können. Eine zukünftige Implementierung könnte Partikel generieren indem die gegnerische Figurenanzahl gespeichert werden und nach dem zusammenführen überprüft wird, ob die gegnerischen Figuren vollständig im Partikel vertreten sind. Wenn nicht, dann könnten die fehlenden Figuren auf Feldern aufgestellt werden, welche die aktuelle Spielerin nicht sehen kann.

Auch wurde in dieser Arbeit auf eine selbst entwickelte Bewertungsfunktion in Dark Pawn Chess und in den anderen beiden Spielen eine angepasste Bewertungsfunktion von Shannon zurückgegriffen. Durch das nutzen der State-of-the-art Bewertungsfunktion Stockfish sollte es möglich sein die Ergebnisse der Hybride weiter zu steigern.

Literaturverzeichnis

- [A⁺94] ALLIS, LOUIS VICTOR et al.: *Searching for solutions in games and artificial intelligence*, Band 14. Ponsen & Looijen Wageningen, 1994.
- [BW14] BAIER, HENDRIK und MARK HM WINANDS: *Monte-carlo tree search and minimax hybrids with heuristic evaluation functions*. In: *Workshop on Computer Games*, Seiten 45–63. Springer, 2014.
- [Cou06] COULOM, RÉMI: *Efficient selectivity and backup operators in Monte-Carlo tree search*. In: *International conference on computers and games*, Seiten 72–83. Springer, 2006.
- [Koh26] KOHLSTÄDT, JÜRGEN: *Die FIDE – Schachregeln*. https://www.schachbund.de/files/dsb/srk/fide-regeln/FIDE-Regeln_2023_ger_20230304.pdf, 2026. Zugriff am 15.02.2026.
- [Lor15] LORENTZ, RICHARD: *Early playout termination in MCTS*. In: *Advances in Computer Games*, Seiten 12–19. Springer, 2015.
- [Ras04] RASMUSSEN, DAVID: *Parallel chess searching and bitboards*. Diplomarbeit, Technical University of Denmark, DTU, DK-2800 Kgs. Lyngby, Denmark, 2004.
- [Sha50] SHANNON, CLAUDE E: XXII. *Programming a computer for playing chess*. The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science, 41(314):256–275, 1950.
- [SV10] SILVER, DAVID und JOEL VENESS: *Monte-Carlo Planning in Large POMDPs*. In: LAFFERTY, J., C. WILLIAMS, J. SHAWE-TAYLOR, R. ZEMEL und A. CULOTTA (Herausgeber): *Advances in Neural Information Processing Systems*, Band 23. Curran Associates, Inc., 2010.
- [WB09] WINANDS, MARK HM und YNGVI BJÖRNSSON: *Evaluation function based monte-carlo LOA*. In: *Advances in Computer Games*, Seiten 33–44. Springer, 2009.
- [Zob90] ZOBRIST, ALBERT L: *A new hashing method with application for game playing*. ICGA Journal, 13(2):69–73, 1990.